DISEÑO DEL MODELO FÍSICO DE BASE DE DATOS

ORALIA DEL SOCORRO CORTÉS GRAJALES



LUIS ROJAS CANO

EMILY QUINTERO RIVERA

SOFIA HIDALGO CÓRDOBA

JEINER ANDRÉS MOSQUERA

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

FACULTAD DE INGENIERÍA

BASES DE DATOS 1

2026

Introducción y Contexto del Proyecto

En el presente informe se expone el diseño e implementación del modelo físico de base de datos orientado a soportar la arquitectura de una red social de uso académico exclusivo para estudiantes. El objetivo principal de este trabajo es transformar las especificaciones lógicas previamente validadas en un esquema físico funcional e integro, garantizando la consistencia de los datos, la ausencia de redundancias innecesarias y el cumplimiento estricto de las reglas de negocio planteadas durante el curso.

Justificación del SGBD Seleccionado

Para la implementación del modelo físico del sistema de red social académica, se seleccionó el sistema gestor de bases de datos PostgreSQL debido a sus sólidas características técnicas y de integridad:

Soporte de estándares SQL ANSI: Permite la definición estricta de dominios, reglas de unicidad UNIQUE, restricciones de validación CHECK y claves primarias compuestas.

Integridad referencial avanzada: Garantiza un control eficiente sobre las operaciones de la base de datos, soportando comportamientos automáticos como eliminaciones en cascada (ON DELETE CASCADE) y restricciones de protección (ON DELETE RESTRICT).

Alineación con el material del curso: La sintaxis de PostgreSQL y su gestión nativa de tipos de datos (TIMESTAMP, VARCHAR, TEXT, DATE, TIME) coinciden completamente con las especificaciones técnicas exigidas en la unidad de estudio.

Decisiones de Modelado y Normalización

El modelo físico desarrollado consta de un total de 18 tablas, las cuales fueron derivadas directamente del modelo lógico validado previamente. Para asegurar un diseño óptimo, eficiente y sin duplicidad de información, se aplicaron rigurosamente las formas normales:

Primera Forma Normal (1FN): Se eliminaron los atributos multivaluados (tales como listas de intereses o cursos), descomponiendo y atomizando la información en tablas independientes.

Segunda Forma Normal (2FN): Se implementó en las 7 tablas intermedias con claves primarias compuestas (estudiante_interes, membresia, seguimiento, inscripcion_evento, participacion_chat, estudiante_curso y estudiante_investigacion), asegurando que todos los atributos descriptivos dependan de manera completa de la combinación de la clave.

Tercera Forma Normal (3FN): Se eliminaron las dependencias transitivas en todo el esquema. De esta manera, los datos del estudiante no se repiten en las publicaciones ni en los comentarios, sino que se referencian únicamente mediante la clave foránea id_estudiante.

Tabla	Propósito	Claves principales
estudiante	Identifica a cada estudiante.	id_estudiante PK; documento UK; correo UK.
perfil	Información complementaria del estudiante.	id_perfil PK; id_estudiante FK y UK.
publicacion	Publicaciones creadas por estudiantes.	id_publicacion PK; id_estudiante FK.
comentario	Comentarios sobre publicaciones.	id_comentario PK; id_publicacion FK; id_estudiante FK.
reaccion	Reacciones de estudiantes a publicaciones.	id_reaccion PK; id_publicacion FK; id_estudiante FK.
interes	Catálogo de intereses.	id_interes PK; nombre UK.
estudiante_interes	Relación N:M entre estudiantes e intereses.	PK compuesta: id_estudiante + id_interes; ambos FK.
grupo	Grupos de estudio o comunidades.	id_grupo PK; nombre UK.
membresia	Pertenencia de estudiantes a grupos.	PK compuesta: id_estudiante + id_grupo; ambos FK.
seguimiento	Relación recursiva entre estudiantes.	PK compuesta: id_seguidor + id_seguido; ambos FK; CHECK anti-autoseguimiento.
evento	Eventos creados por estudiantes.	id_evento PK; id_creador FK.
inscripcion_evento	Relación N:M entre estudiantes y eventos.	PK compuesta: id_estudiante + id_evento; ambos FK.
chat	Conversaciones o espacios de chat.	id_chat PK.
participacion_chat	Participación de estudiantes en chats.	PK compuesta: id_estudiante + id_chat; ambos FK.
curso_compartido	Catálogo de cursos compartidos.	codigo_curso PK.
estudiante_curso	Relación N:M entre estudiantes y cursos.	PK compuesta: id_estudiante + codigo_curso; ambos FK.
Investigacion	Iniciativas o temas de investigación.	id_investigacion PK.
estudiante_investigacion	Relación N:M entre estudiantes e investigaciones.	PK compuesta: id_estudiante + id_investigacion; ambos FK.

Descripción Detallada de las Tablas del Modelo Físico

A continuación, se describen las 18 tablas que conforman el sistema, agrupadas por módulos funcionales para facilitar la comprensión de la arquitectura relacional:

1. Módulo de Usuario y Perfil

estudiante: Es la tabla principal del sistema. Almacena los datos de identificación y de contacto de cada alumno registrado (id_estudiante, nombre, apellido, documento, correo y carrera).

perfil: Entidad que extiende la información del usuario mediante una relación 1:1 con estudiante. Contiene elementos opcionales como la fotografía (foto) y una descripción biográfica o presentación (descripcion).

seguimiento: Tabla intermedia que gestiona la relación recursiva N:M de seguimiento entre alumnos. Contiene las claves id_seguidor e id_seguido, junto con la fecha_seguimiento.

```
CREATE TABLE estudiante (
    id_estudiante SERIAL PRIMARY KEY,
    nombre VARCHAR(60) NOT NULL,
    apellido VARCHAR(60) NOT NULL,
    documento VARCHAR(20) NOT NULL UNIQUE,
    correo VARCHAR(120) NOT NULL UNIQUE,
    carrera VARCHAR(100) NOT NULL
);

CREATE TABLE perfil (
    id_perfil SERIAL PRIMARY KEY,
    id_estudiante INT NOT NULL UNIQUE REFERENCES estudiante(id_estudiante)
ON DELETE CASCADE,
    foto VARCHAR(255),
    descripcion VARCHAR(500)
);
```

2. Módulo de Intereses y Comunidades

interes: Funciona como catálogo general de áreas del conocimiento o aficiones (nombre, tipo) que los usuarios pueden vincular a su perfil.

estudiante_interes: Tabla intermedia N:M que asocia a los estudiantes con sus temas de interés elegidos.

grupo: Representa los grupos de estudio o comunidades formadas dentro de la plataforma, almacenando su nombre y descripción.

membresia: Tabla intermedia N:M entre estudiante y grupo. Registra la fecha_ingreso y el atributo rol_grupo para asignar funciones de moderación a determinados miembros.

```
-- 2. MÓDULO DE INTERESES Y HABILIDADES
CREATE TABLE interes (
    id_interes SERIAL PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL UNIQUE,
    tipo VARCHAR(40) NOT NULL
);

CREATE TABLE estudiante_interes (
```

```
    id_estudiante INT REFERENCES estudiante(id_estudiante) ON DELETE
CASCADE,
    id_interes INT REFERENCES interes(id_interes) ON DELETE CASCADE,
    PRIMARY KEY (id_estudiante, id_interes)
);
```

3. Módulo de Comunicación y Chats

chat: Almacena las salas o canales de conversación (grupales o individuales), registrando el tipo_chat, nombre y fecha_creacion.

participacion_chat: Tabla intermedia N:M que conecta a cada usuario con las salas de chat en las que está integrado, incluyendo su fecha_ingreso.

```
-- 3. MÓDULO DE INTERACCIÓN Y CONTENIDO
CREATE TABLE publicacion (
    id_publicacion SERIAL PRIMARY KEY,
    id_estudiante INT NOT NULL REFERENCES estudiante(id_estudiante) ON
DELETE CASCADE,
    titulo VARCHAR(150) NOT NULL,
    contenido TEXT NOT NULL,
    fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    tipo_publicacion VARCHAR(40) NOT NULL
);

CREATE TABLE comentario (
    id_comentario SERIAL PRIMARY KEY,
    id_publicacion INT NOT NULL REFERENCES publicacion(id_publicacion) ON
DELETE CASCADE,
    id_estudiante INT NOT NULL REFERENCES estudiante(id_estudiante) ON
DELETE CASCADE,
    contenido TEXT NOT NULL,
    fecha_comentario TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE reaccion (
    id_reaccion SERIAL PRIMARY KEY,
    id_publicacion INT NOT NULL REFERENCES publicacion(id_publicacion) ON
DELETE CASCADE,
    id_estudiante INT NOT NULL REFERENCES estudiante(id_estudiante) ON
DELETE CASCADE,
    tipo_reaccion VARCHAR(30) NOT NULL,
```

```
    CONSTRAINT uk_reaccion_estudiante_pub UNIQUE (id_estudiante,
id_publicacion)
);
```

4. Módulo de Contenidos e Interacción

publicacion: Registra las publicaciones compartidas por los usuarios, guardando el título, contenido, fecha_publicacion, tipo_publicacion y la referencia al autor mediante id_estudiante.

comentario: Guarda las respuestas u opiniones de los estudiantes en las publicaciones, vinculando el id_publicacion con el id_estudiante del autor.

reaccion: Registra las interacciones (como "me gusta" u otros tipos de reacción) realizadas por los estudiantes en las publicaciones.

```
-- 4. MÓDULO DE COMUNIDAD Y RED SOCIAL
CREATE TABLE grupo (
    id_grupo SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    descripcion VARCHAR(500)
);

CREATE TABLE membresia (
    id_estudiante INT REFERENCES estudiante(id_estudiante) ON DELETE
CASCADE,
    id_grupo INT REFERENCES grupo(id_grupo) ON DELETE CASCADE,
    fecha_ingreso TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    rol_grupo VARCHAR(30) DEFAULT 'Miembro' CHECK (rol_grupo IN ('Miembro',
'Moderador', 'Líder')),
    PRIMARY KEY (id_estudiante, id_grupo)
);

CREATE TABLE seguimiento (
    id_seguidor INT REFERENCES estudiante(id_estudiante) ON DELETE CASCADE,
    id_seguido INT REFERENCES estudiante(id_estudiante) ON DELETE CASCADE,
    fecha_seguimiento DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (id_seguidor, id_seguido),
    CONSTRAINT chk_no_autoseguimiento CHECK (id_seguidor <> id_seguido)
);
```

5. Módulo de Eventos

evento: Almacena la información de eventos creados por la comunidad (nombre, fecha, hora, lugar, tipo_evento, estado_evento) y guarda la referencia de su creador (id_creador).

inscripcion_evento: Tabla intermedia N:M que gestiona el registro de los estudiantes a los distintos eventos, guardando la fecha_inscripcion y el estado_inscripcion.

```
-- 5. MÓDULO DE ACTIVIDADES Y EVENTOS
CREATE TABLE evento (
    id_evento SERIAL PRIMARY KEY,
    id_creador INT NOT NULL REFERENCES estudiante(id_estudiante) ON DELETE
RESTRICT,
    nombre VARCHAR(120) NOT NULL,
    descripcion VARCHAR(500),
    fecha DATE NOT NULL,
    hora TIME NOT NULL,
    lugar VARCHAR(150) NOT NULL,
    tipo_evento VARCHAR(50) NOT NULL,
    estado_evento VARCHAR(30) DEFAULT 'Planeado' CHECK (estado_evento IN
('Planeado', 'Activo', 'Finalizado', 'Cancelado'))
);

CREATE TABLE inscripcion_evento (
    id_estudiante INT REFERENCES estudiante(id_estudiante) ON DELETE
CASCADE,
    id_evento INT REFERENCES evento(id_evento) ON DELETE CASCADE,
    fecha_inscripcion DATE DEFAULT CURRENT_DATE,
    estado_inscripcion VARCHAR(30) DEFAULT 'Inscrito',
    PRIMARY KEY (id_estudiante, id_evento)
);
```

6. Módulo Académico y de Investigación

curso_compartido: Contiene el catálogo de asignaturas o cursos de las facultades (codigo_curso, nombre, facultad, nivel_complejidad).

estudiante_curso: Tabla intermedia N:M que relaciona a los estudiantes con los cursos en los que están inscritos o que comparten.

investigacion: Registra los proyectos de investigación o semilleros disponibles (nombre, titulo, publico_interes).

estudiante_investigacion: Tabla intermedia N:M que vincula a los estudiantes participantes con sus respectivos proyectos de investigación.

```
-- 6. MÓDULO DE COMUNICACIÓN
CREATE TABLE chat (
    id_chat SERIAL PRIMARY KEY,
```

```
    tipo_chat VARCHAR(20) NOT NULL CHECK (tipo_chat IN ('Individual',
'Grupal')),
    nombre_chat VARCHAR(100),
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE participacion_chat (
    id_estudiante INT REFERENCES estudiante(id_estudiante) ON DELETE
CASCADE,
    id_chat INT REFERENCES chat(id_chat) ON DELETE CASCADE,
    fecha_ingreso TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_estudiante, id_chat)
);


CREATE TABLE curso_compartido (
    codigo_curso VARCHAR(20) PRIMARY KEY,
    nombre_curso VARCHAR(120) NOT NULL,
    facultad VARCHAR(100) NOT NULL,
    nivel_complejidad VARCHAR(30)
);

CREATE TABLE estudiante_curso (
    id_estudiante INT REFERENCES estudiante(id_estudiante) ON DELETE
CASCADE,
    codigo_curso VARCHAR(20) REFERENCES curso_compartido(codigo_curso) ON
DELETE CASCADE,
    PRIMARY KEY (id_estudiante, codigo_curso)
);

CREATE TABLE investigacion (
    id_investigacion SERIAL PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL,
    titulo VARCHAR(200) NOT NULL,
    publico_interes VARCHAR(200)
);

CREATE TABLE estudiante_investigacion (
    id_estudiante INT REFERENCES estudiante(id_estudiante) ON DELETE
CASCADE,
    id_investigacion INT REFERENCES investigacion(id_investigacion) ON
DELETE CASCADE,
    PRIMARY KEY (id_estudiante, id_investigacion)
);
```

Reglas de Integridad y Control de Alcance

Con el fin de garantizar la coherencia de los datos y respetar los límites definidos para el proyecto, se aplicaron las siguientes reglas de negocio e integridad:

Exclusividad estudiantil: Dado que el sistema está diseñado únicamente para estudiantes, se descartó la creación de entidades para docentes o administradores. Las funciones de moderación se resuelven mediante la columna rol_grupo en la tabla membresia.

Relación 1:1 (estudiante - perfil): Se asegura que cada estudiante disponga de un solo perfil mediante las restricciones REFERENCES estudiante(id_estudiante) y la regla de unicidad UNIQUE sobre la clave foránea en la tabla perfil.

Relación recursiva N:M (seguimiento): Para evitar inconsistencias en el seguimiento entre pares, se estableció la restricción CHECK (id_seguidor <> id_seguido), impidiendo que un estudiante pueda seguirse a sí mismo.

Tipos de datos y Justificación

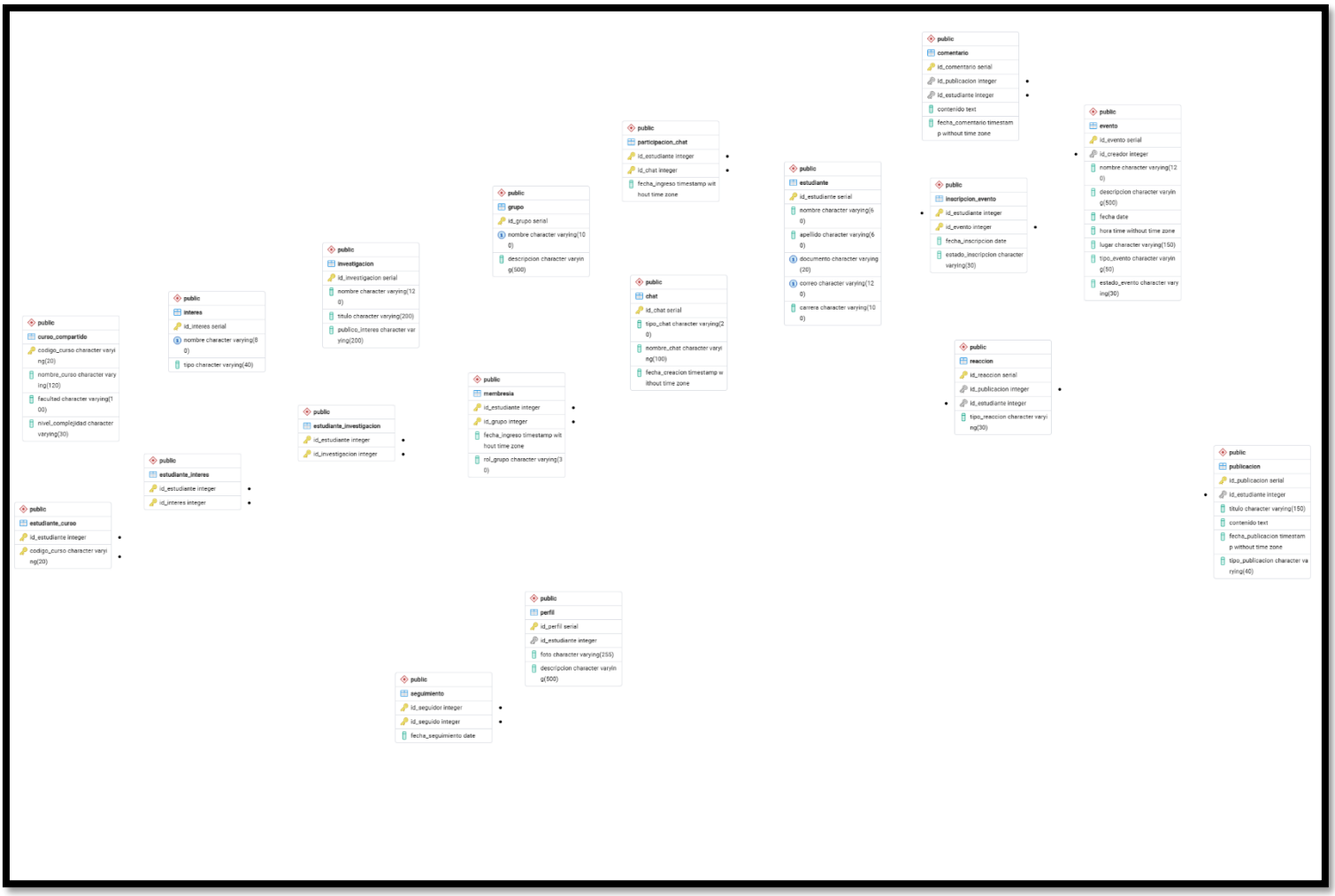
Tipo de dato	Uso en el proyecto	Justificación
INTEGER IDENTITY	Identificadores como id_estudiante o id_publicacion.	Genera identificadores numéricos automáticamente.
VARCHAR(n)	Nombres, apellidos, correo, documento, títulos y categorías.	Limita la longitud cuando se conoce un máximo razonable.
TEXT	Contenido de publicaciones y comentarios.	Permite contenido textual extenso y variable.
DATE	Fecha de ingreso, inscripción o evento.	Guarda fecha sin componente horario.
TIME	Hora del evento.	Guarda la hora separada de la fecha.
TIMESTAMP	Fecha de creación o publicación con hora.	Registra fecha y hora en un mismo campo.

Las longitudes VARCHAR son decisiones de diseño iniciales y se mantienen consistentes con el modelo lógico, además, los campos declarados NOT NULL son obligatorios y los campos opcionales, como foto o descripción de perfil, pueden aceptar NULL.

Es importante aclarar que si tenemos un NULL en la metadata de longitud (espacio que ocupa un dato) o de valor predeterminado no significa que la tabla esté dañada: para algunos tipos de datos la longitud de caracteres no aplica, y un campo puede no tener valor predeterminado aunque sea obligatorio.

Diagrama Entidad-Relación Físico (ERD)

El esquema físico completo de la base de datos con sus 18 tablas y sus relaciones se encuentra representado visualmente en el Diagrama Entidad-Relación (ERD) del proyecto.



Conclusiones del Aprendizaje

La elaboración de este modelo me permitió entender la transición entre un modelo conceptual y uno físico, analizando la forma en que las relaciones de muchos a muchos se transforman de manera correcta en tablas intermedias mediante claves foráneas compuestas.

La aplicación metódica de la 1FN, 2FN y 3FN me enseñó la importancia de estructurar una base de datos limpia, reduciendo la redundancia al mínimo y previniendo anomalías en las operaciones de actualización o eliminación.

Comprendí cómo tomar decisiones de diseño orientadas al negocio, solucionando los permisos de moderación con atributos en las relaciones de membresía sin la necesidad de añadir tablas administrativas innecesarias.

El modelo físico alcanzado organiza el sistema en módulos funcionales bien definidos, garantizando la integridad referencial para asegurar un comportamiento estable durante la ejecución de sentencias DML.